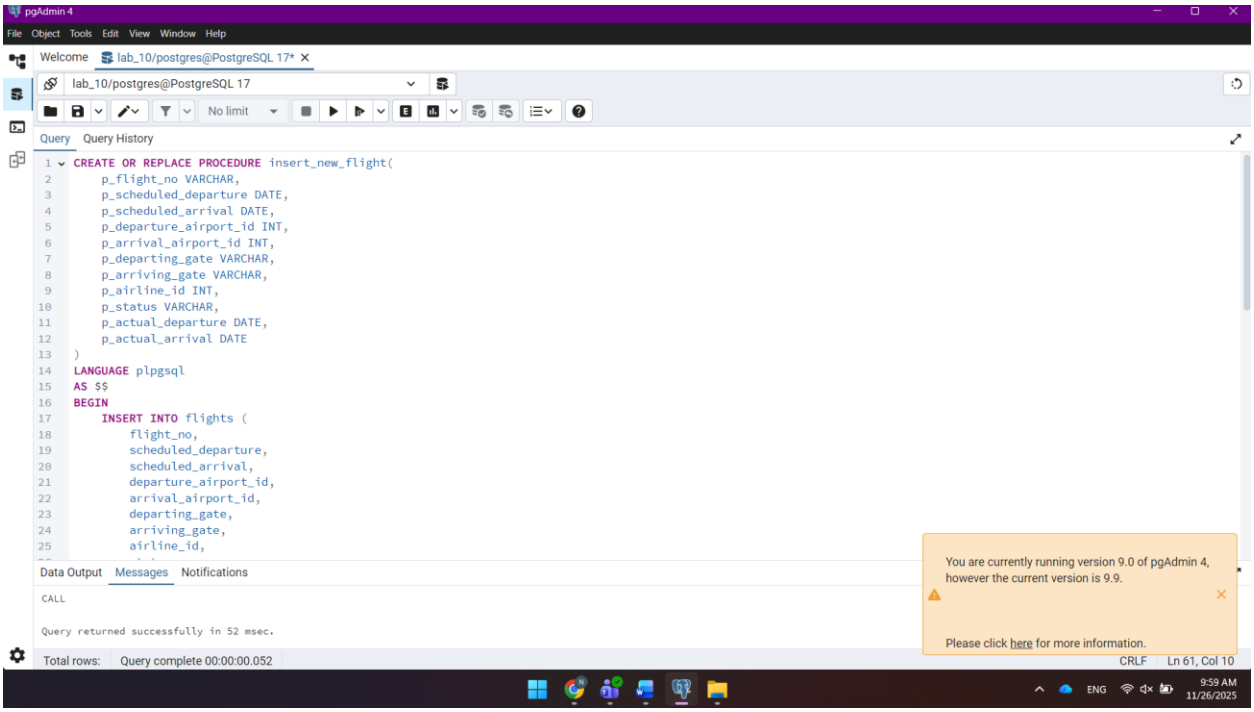
Lab_10

1




```
CREATE OR REPLACE PROCEDURE insert_new_flight(
    p_flight_no VARCHAR,
    p_scheduled_departure DATE,
    p_scheduled_arrival DATE,
    p_departure_airport_id INT,
    p_arrival_airport_id INT,
    p_departing_gate VARCHAR,
    p_arriving_gate VARCHAR,
    p_airline_id INT,
```

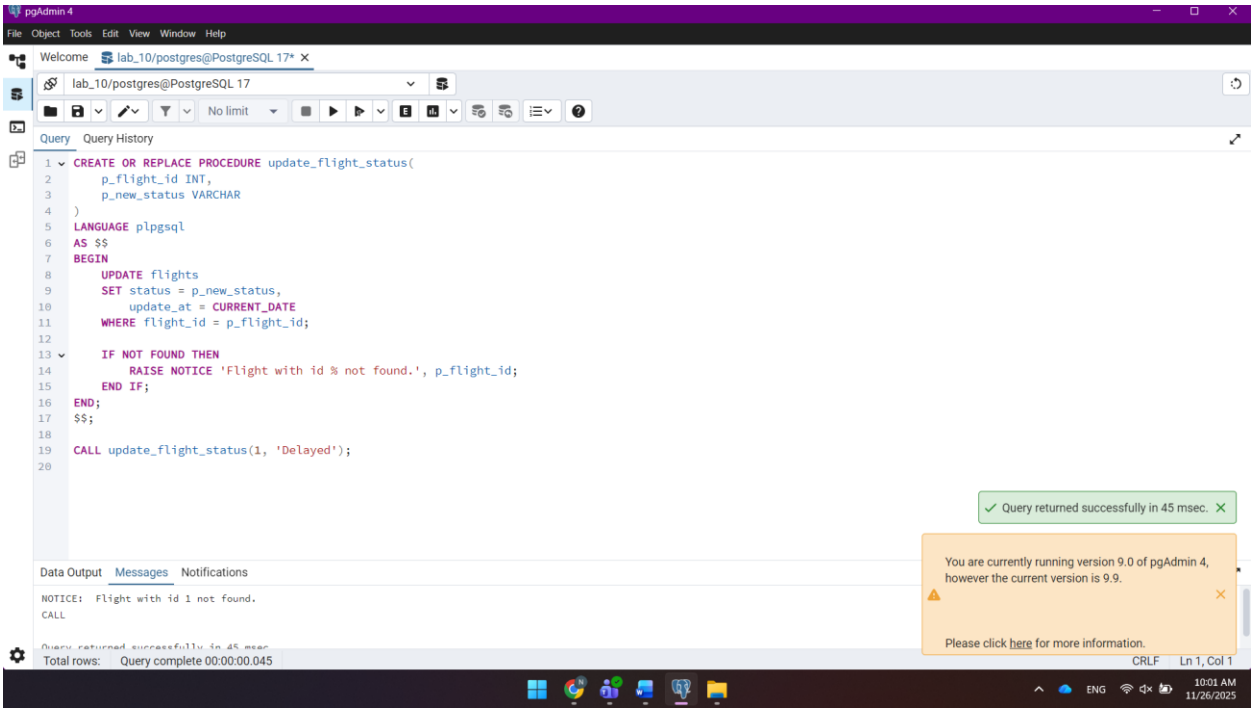
```
    p_status VARCHAR,
    p_actual_departure DATE,
    p_actual_arrival DATE
)
LANGUAGE plpgsql
AS $$
BEGIN
    INSERT INTO flights (
        flight_no,
        scheduled_departure,
        scheduled_arrival,
        departure_airport_id,
        arrival_airport_id,
        departing_gate,
        arriving_gate,
        airline_id,
        status,
        actual_departure,
        actual_arrival,
        created_at,
        update_at
    )
    VALUES (
        p_flight_no,
```

```
        p_scheduled_departure,
        p_scheduled_arrival,
        p_departure_airport_id,
        p_arrival_airport_id,
        p_departing_gate,
        p_arriving_gate,
        p_airline_id,
        p_status,
        p_actual_departure,
        p_actual_arrival,
        CURRENT_DATE,
        CURRENT_DATE
    );
END;
$$;

CALL insert_new_flight(
    'KC123',
    '2025-12-01',
    '2025-12-01',
    1,
    2,
    'A5',
    'B3',
```

```
    1,
    'Scheduled',
    NULL,
    NULL
);
```

2



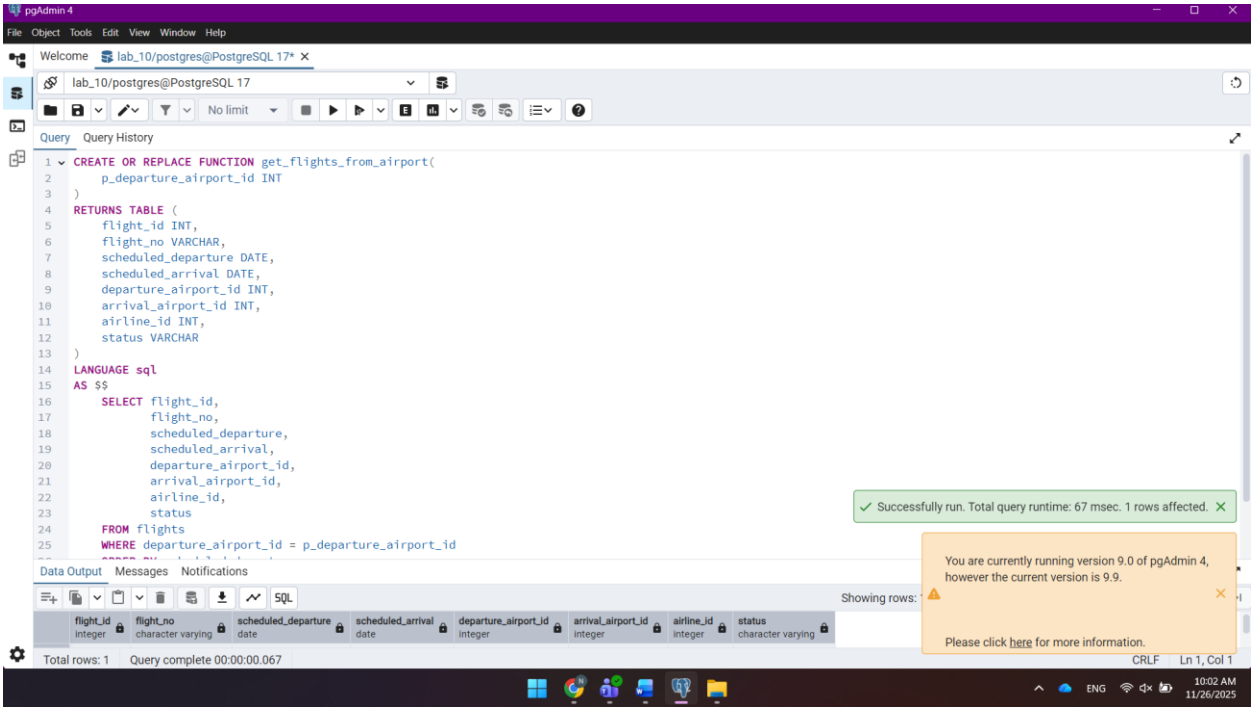

```
CREATE OR REPLACE PROCEDURE update_flight_status(
    p_flight_id INT,
    p_new_status VARCHAR
)
LANGUAGE plpgsql
AS $$
BEGIN
```

```
    UPDATE flights
    SET status = p_new_status,
        update_at = CURRENT_DATE
    WHERE flight_id = p_flight_id;

    IF NOT FOUND THEN
        RAISE NOTICE 'Flight with id % not found.', p_flight_id;
    END IF;
END;
$$;

CALL update_flight_status(1, 'Delayed');
```

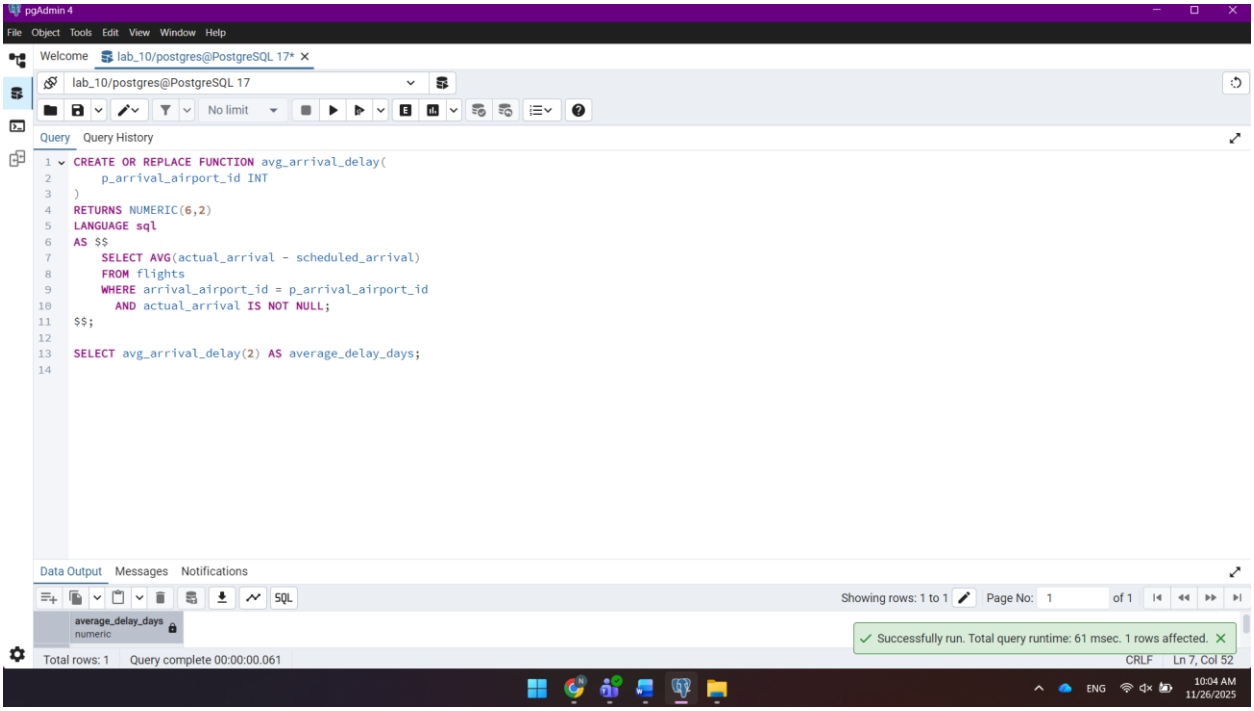
3




```
CREATE OR REPLACE FUNCTION get_flights_from_airport(
    p_departure_airport_id INT
)
RETURNS TABLE (
    flight_id INT,
    flight_no VARCHAR,
    scheduled_departure DATE,
    scheduled_arrival DATE,
    departure_airport_id INT,
    arrival_airport_id INT,
    airline_id INT,
    status VARCHAR
)
LANGUAGE sql
```

```
AS $$
   SELECT flight_id,
         flight_no,
         scheduled_departure,
         scheduled_arrival,
         departure_airport_id,
         arrival_airport_id,
         airline_id,
         status
   FROM flights
   WHERE departure_airport_id = p_departure_airport_id
   ORDER BY scheduled_departure;
$$;
SELECT * FROM get_flights_from_airport(1);
```

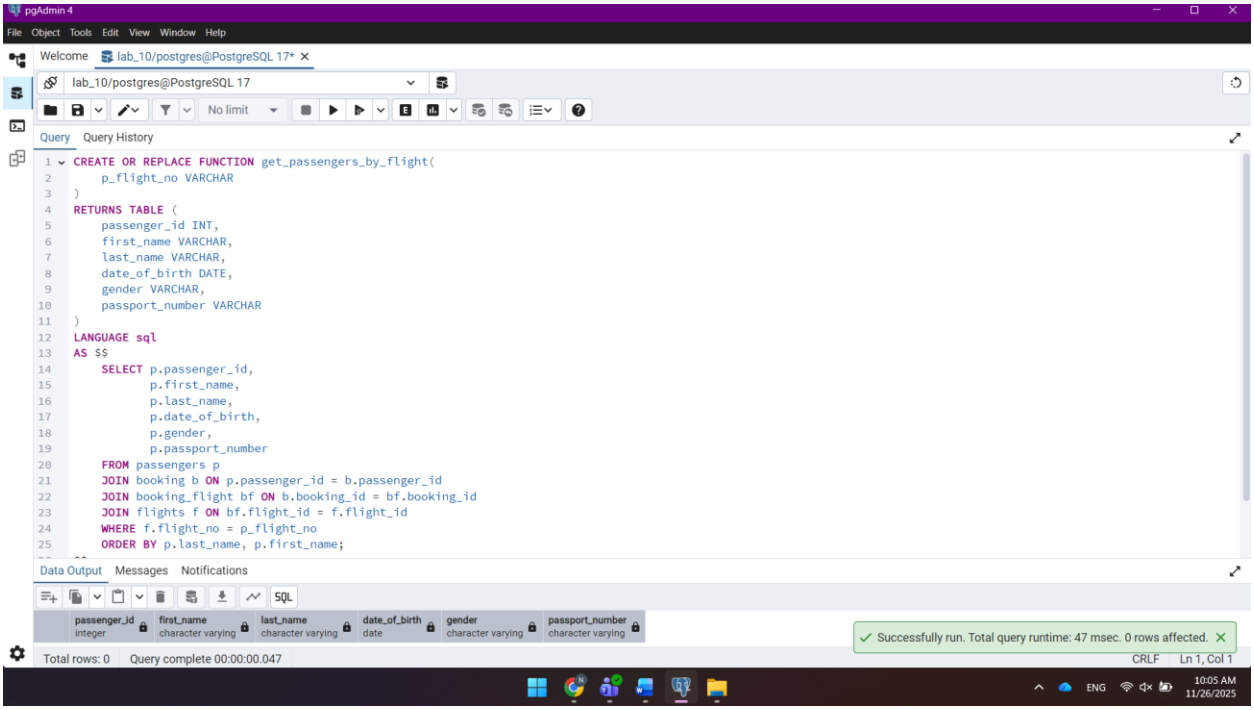
4




```
CREATE OR REPLACE FUNCTION avg_arrival_delay(
    p_arrival_airport_id INT
)
RETURNS NUMERIC(6,2)
LANGUAGE sql
AS $$
    SELECT AVG(actual_arrival - scheduled_arrival)
    FROM flights
    WHERE arrival_airport_id = p_arrival_airport_id
      AND actual_arrival IS NOT NULL;
$$;

SELECT avg_arrival_delay(2) AS average_delay_days;
```

5

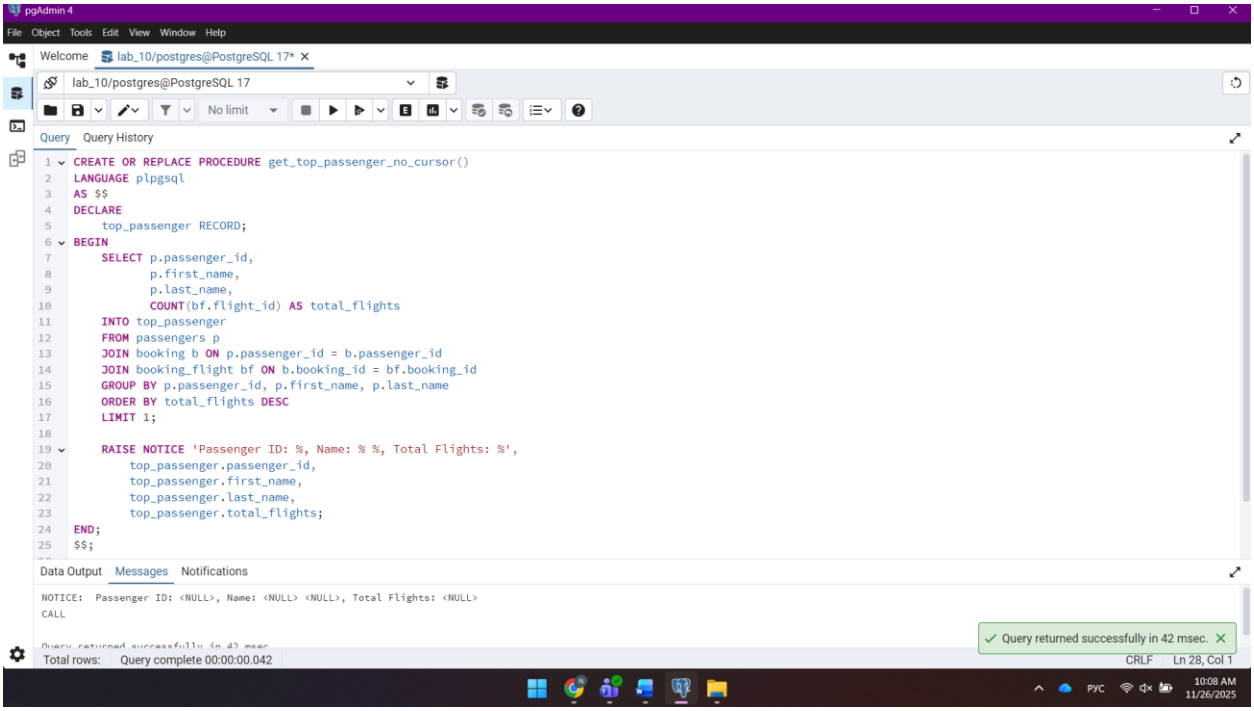



```
CREATE OR REPLACE FUNCTION get_passengers_by_flight(
    p_flight_no VARCHAR
)
RETURNS TABLE (
    passenger_id INT,
    first_name VARCHAR,
    last_name VARCHAR,
    date_of_birth DATE,
    gender VARCHAR,
    passport_number VARCHAR
)
LANGUAGE sql
AS $$
```

```
    SELECT p.passenger_id,
           p.first_name,
           p.last_name,
           p.date_of_birth,
           p.gender,
           p.passport_number
    FROM passengers p
    JOIN booking b ON p.passenger_id = b.passenger_id
    JOIN booking_flight bf ON b.booking_id = bf.booking_id
    JOIN flights f ON bf.flight_id = f.flight_id
    WHERE f.flight_no = p_flight_no
    ORDER BY p.last_name, p.first_name;
$$;

SELECT * FROM get_passengers_by_flight('KC123');
```

6

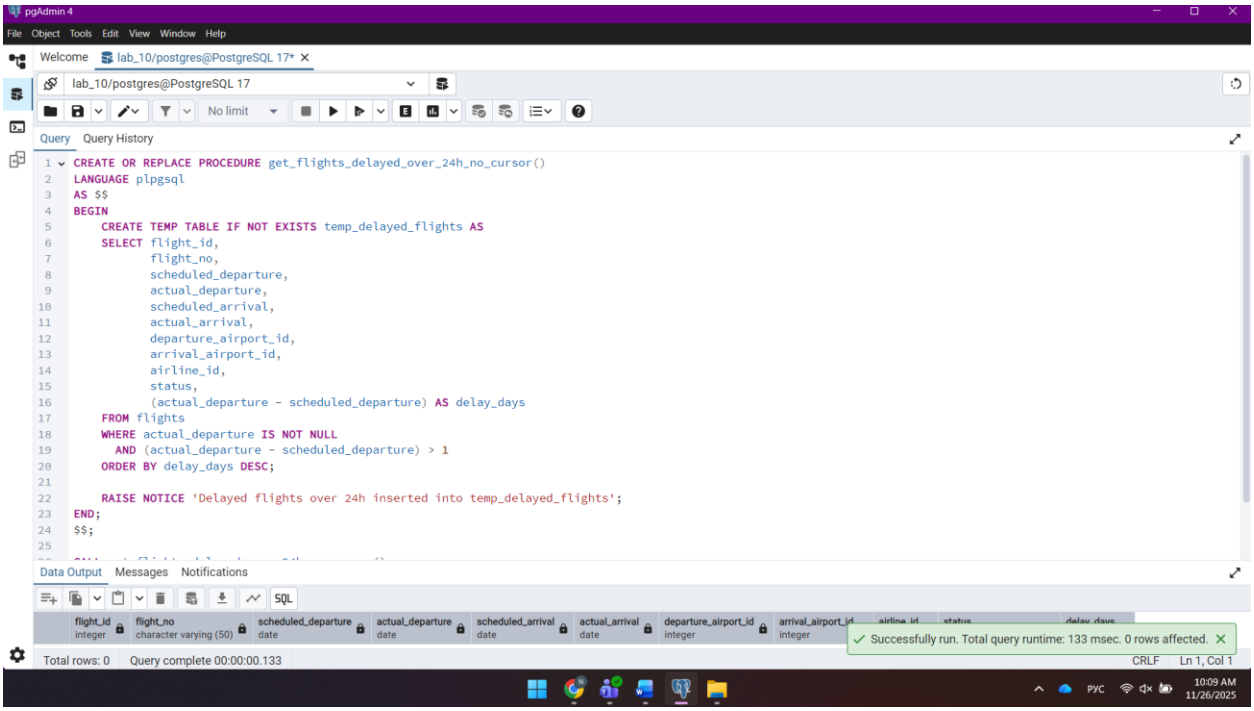



```
CREATE OR REPLACE PROCEDURE get_top_passenger_no_cursor()
LANGUAGE plpgsql
AS $$
DECLARE
    top_passenger RECORD;
BEGIN
    SELECT p.passenger_id,
           p.first_name,
           p.last_name,
           COUNT(bf.flight_id) AS total_flights
    INTO top_passenger
    FROM passengers p
    JOIN booking b ON p.passenger_id = b.passenger_id
    JOIN booking_flight bf ON b.booking_id = bf.booking_id
```

```
    GROUP BY p.passenger_id, p.first_name, p.last_name
    ORDER BY total_flights DESC
    LIMIT 1;

    RAISE NOTICE 'Passenger ID: %, Name: % %, Total Flights: %',
        top_passenger.passenger_id,
        top_passenger.first_name,
        top_passenger.last_name,
        top_passenger.total_flights;
END;
$$;

CALL get_top_passenger_no_cursor();
```




```
CREATE OR REPLACE PROCEDURE get_flights_delayed_over_24h_no_cursor()
LANGUAGE plpgsql
AS $$
BEGIN
    CREATE TEMP TABLE IF NOT EXISTS temp_delayed_flights AS
    SELECT flight_id,
           flight_no,
           scheduled_departure,
           actual_departure,
           scheduled_arrival,
           actual_arrival,
           departure_airport_id,
           arrival_airport_id,
           airline_id,
           status,
           (actual_departure - scheduled_departure) AS delay_days
    FROM flights
    WHERE actual_departure IS NOT NULL
      AND (actual_departure - scheduled_departure) > 1
    ORDER BY delay_days DESC;

    RAISE NOTICE 'Delayed flights over 24h inserted into temp_delayed_flights';
END;
$$;
```

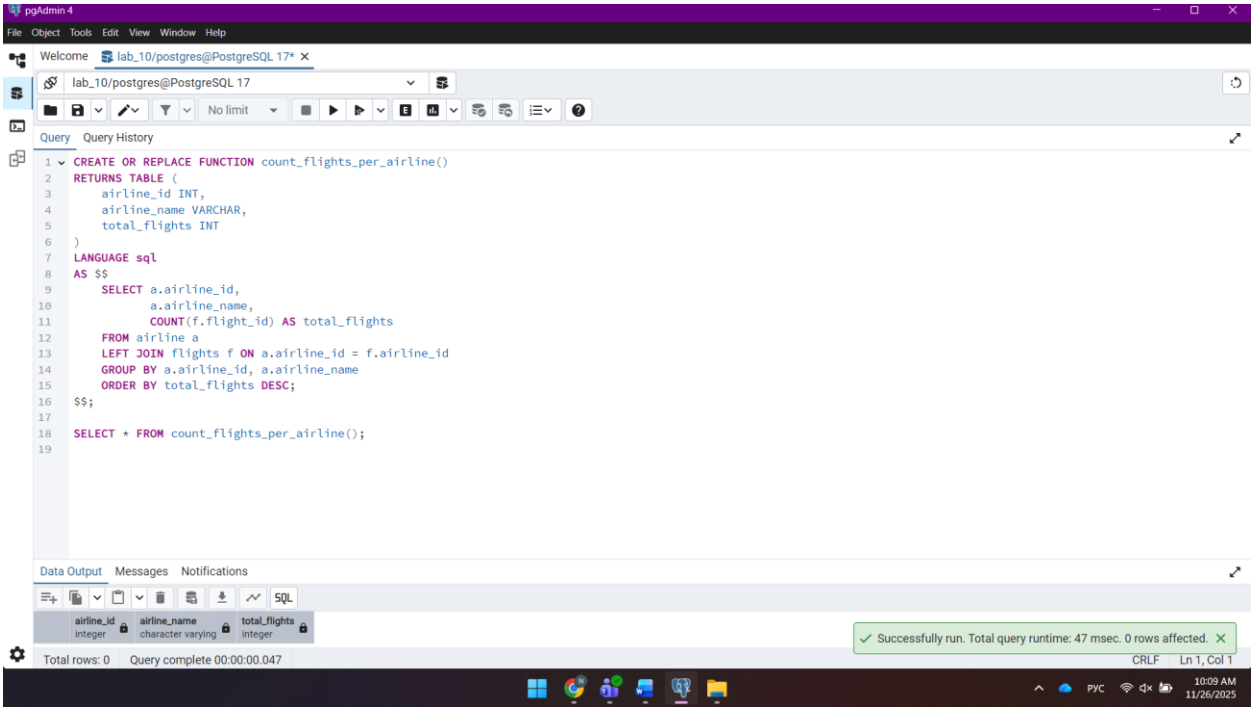
```
        status,
        (actual_departure - scheduled_departure) AS delay_days
    FROM flights
    WHERE actual_departure IS NOT NULL
      AND (actual_departure - scheduled_departure) > 1
    ORDER BY delay_days DESC;

    RAISE NOTICE 'Delayed flights over 24h inserted into
temp_delayed_flights';
END;
$$;

CALL get_flights_delayed_over_24h_no_cursor();

SELECT * FROM temp_delayed_flights;
```

8




```
CREATE OR REPLACE FUNCTION count_flights_per_airline()
RETURNS TABLE (
    airline_id INT,
    airline_name VARCHAR,
    total_flights INT
)
LANGUAGE sql
AS $$
    SELECT a.airline_id,
           a.airline_name,
           COUNT(f.flight_id) AS total_flights
    FROM airline a
    LEFT JOIN flights f ON a.airline_id = f.airline_id
```

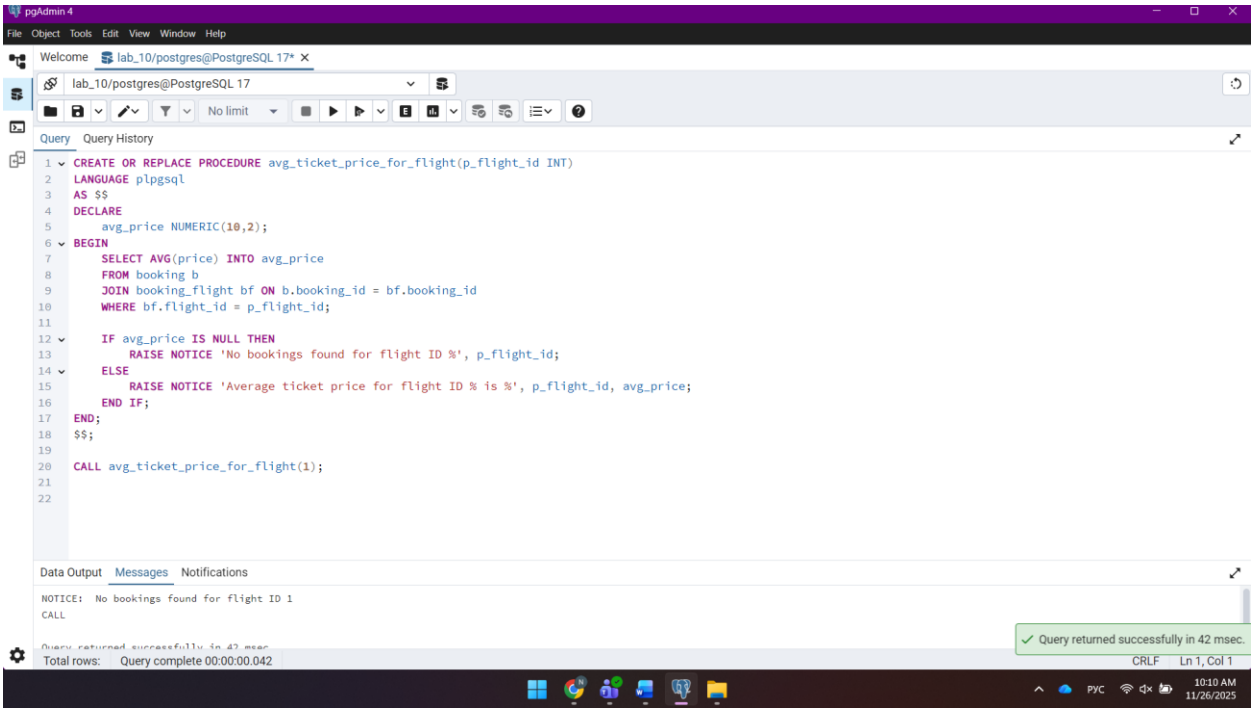
GROUP BY a.airline_id, a.airline_name

ORDER BY total_flights DESC;

$$;

SELECT * FROM count_flights_per_airline();

9



CREATE OR REPLACE PROCEDURE avg_ticket_price_for_flight(p_flight_id INT)

LANGUAGE plpgsql

AS $$

DECLARE

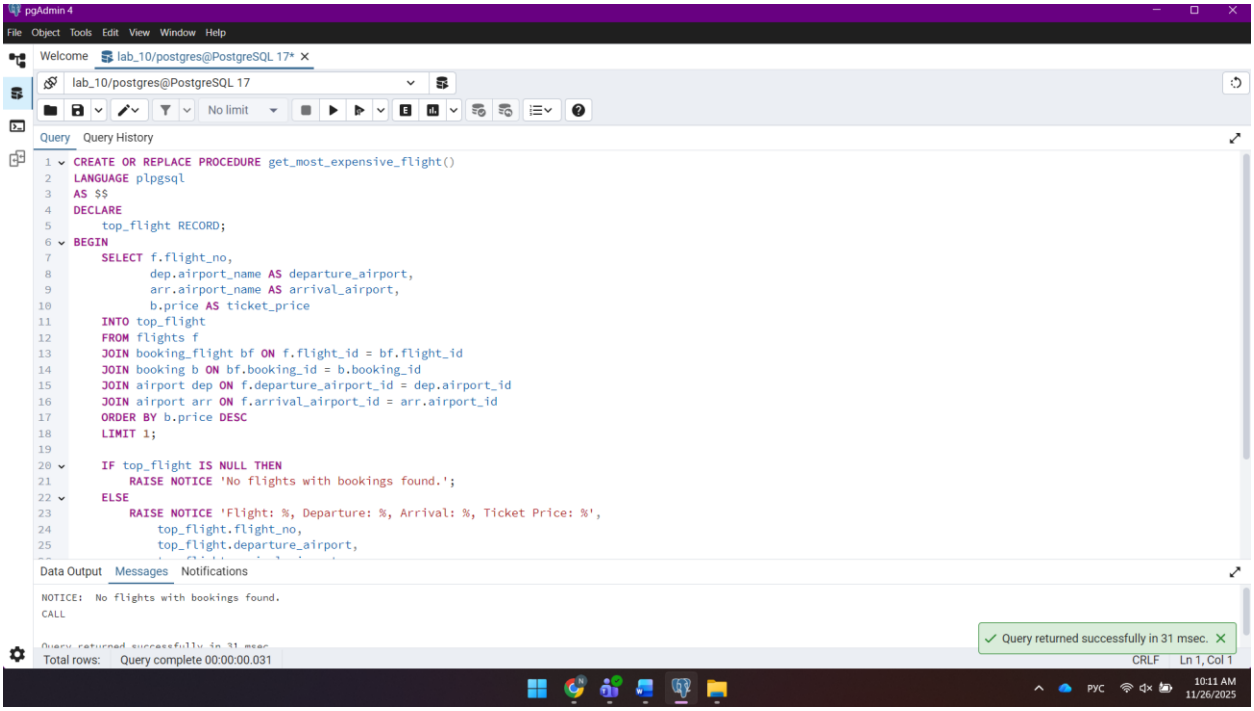
avg_price NUMERIC(10,2);

BEGIN

```
    SELECT AVG(price) INTO avg_price
    FROM booking b
    JOIN booking_flight bf ON b.booking_id = bf.booking_id
    WHERE bf.flight_id = p_flight_id;

    IF avg_price IS NULL THEN
        RAISE NOTICE 'No bookings found for flight ID %', p_flight_id;
    ELSE
        RAISE NOTICE 'Average ticket price for flight ID % is %',
p_flight_id, avg_price;
    END IF;
END;
$$;

CALL avg_ticket_price_for_flight(1);
```




```
CREATE OR REPLACE PROCEDURE get_most_expensive_flight()
LANGUAGE plpgsql
AS $$
DECLARE
    top_flight RECORD;
BEGIN
    SELECT f.flight_no,
           dep.airport_name AS departure_airport,
           arr.airport_name AS arrival_airport,
           b.price AS ticket_price
    INTO top_flight
    FROM flights f
    JOIN booking_flight bf ON f.flight_id = bf.flight_id
```

```
    JOIN booking b ON bf.booking_id = b.booking_id
    JOIN airport dep ON f.departure_airport_id = dep.airport_id
    JOIN airport arr ON f.arrival_airport_id = arr.airport_id
    ORDER BY b.price DESC
    LIMIT 1;

    IF top_flight IS NULL THEN
        RAISE NOTICE 'No flights with bookings found.';
    ELSE
        RAISE NOTICE 'Flight: %, Departure: %, Arrival: %, Ticket Price:
%',
            top_flight.flight_no,
            top_flight.departure_airport,
            top_flight.arrival_airport,
            top_flight.ticket_price;
    END IF;
END;
$$;

CALL get_most_expensive_flight();
```